Lab 2B: Your Project Brain

Duration: 30-45 min | **Difficulty:** Beginner → Intermediate

After: CLAUDE.md & Memory Architecture slides

Continues from: Lab 2A (your Business Dashboard should be running)

Goal

Teach Claude your team's conventions so every future change follows your standards automatically — without you asking.

Key insight: CLAUDE.md is policy, not documentation. It doesn't describe your project — it controls Claude's behavior.

Steps

1. Clear Context

In your Claude Code session:

```
/clear
```

This starts a fresh context window. The project files are still on disk — Claude just starts with a clean conversation.

2. Create CLAUDE.md

Tell Claude:

```
Create a CLAUDE.md file in the project root with these conventions:

# Business Dashboard - Project Conventions

## Tech Stack
- Node.js + Express backend
- SQLite via better-sqlite3 (no external DB required)
- Vanilla HTML/CSS/JS frontend (no frameworks)
- Dark theme UI: background #0f172a, cards #1e293b, borders #334155

## Code Style
- Use const/let, never var
- Always prefer async/await over callbacks
- Return early for validation failures (guard clauses)
- All routes must use try/catch with proper error responses
- All API responses return JSON
- Error responses include { error: "message" }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found
- SQL uses parameterized queries (never string concatenation)

## Git Conventions
- Conventional commits: feat:, fix:, docs:, refactor:
- One logical change per commit
- No committing node_modules or .db files

## File Structure
- src/ for server code
- public/ for frontend files
- Database file: dashboard.db (auto-created on first run)

## Testing
- Test API endpoints after every change
- Verify frontend loads and displays data after changes
```

3. Verify Claude Reads It

Start a new context to pick up the CLAUDE.md:

```
/clear
```

Then ask:

```
What conventions does this project follow?
```

Expected output: Claude recites your rules back to you — code style, response format, Git conventions, file structure. This confirms CLAUDE.md is loaded.

4. Test Convention Enforcement

Now ask Claude to make a change:

```
Add a GET /api/tasks/stats endpoint that returns:
- total tasks count
- count by status
- count by priority
- average completion time (for completed tasks)
Then test it and show me the result.
```

Watch Claude follow your conventions automatically: - Uses `const` (not `var`) — your code style rule - Returns early for edge cases — your guard clause rule - Wraps in try/catch — your error handling rule - Returns proper JSON with error handling — your API rule - Uses parameterized SQL — your security rule

5. Commit

```
Commit this change following our git conventions
```

Claude should use `feat: add task statistics endpoint` or similar.

> ***You may see a Skills prompt again.*** *Select* ***Yes*** *— Claude is using a workflow skill for the commit.*

What Just Happened?

Every time Claude starts working, it automatically: 1. Looks for `CLAUDE.md` in the project root 2. Reads it before doing anything else 3. Treats it as the source of truth for your project

You wrote ONE file with your conventions, and now: - Every endpoint follows your response format - Every function uses async/await - Every route has try/catch - File placement matches your structure

Without CLAUDE.md: You'd have to remind Claude every time.

With CLAUDE.md: Claude just knows.

> ***The mental shift:*** *You didn't ask for try/catch. You didn't ask for guard clauses. CLAUDE.md made those automatic. That's the difference between a prompt and a policy.*

Add a Convention and Watch It Take Effect

Tell Claude to add a new convention:

```
Add a Logging section to CLAUDE.md with these rules:
- All routes must log request method, path, and response time to console
- Format: [timestamp] METHOD /path - STATUS (XXXms)
```

Clear context so Claude picks up the change, and test it:

```
/clear
```

```
Add a GET /api/health endpoint that returns the server status and uptime
```

Check the response. Does it include logging? Claude should follow the new convention immediately.

Optional: Go Deeper

Add a Testing convention:

```
Add a Testing section to CLAUDE.md: After creating any new endpoint,
Claude must automatically test it with at least 2 test cases
(one success, one error case) and report the results.
```

Clear context and add another endpoint. Does Claude test it automatically?

Try conflicting conventions:

```
Add this rule to CLAUDE.md under Code Style:
"All functions must use callback style, never async/await"
```

Clear context, then add an endpoint. What does Claude do when two rules conflict? (This teaches that CLAUDE.md order matters.)

Create a CLAUDE.md for YOUR real project:

Think about your actual team's coding standards. Start drafting a CLAUDE.md you'd actually use at work. What conventions would you include? What rules do you wish were automatic?

□Lab 2B Checkpoint

- [ ] CLAUDE.md file exists in project root
- [ ] Claude follows your conventions automatically (const, try/catch, guard clauses)
- [ ] Stats endpoint works
- [ ] Logging convention added and working
- [ ] Clean conventional commits

> ***Keep Claude running. Return to slides when your instructor is ready.***
